Smart Food Service System

A PROJECT REPORT

Submitted by

Arun Gladson S
(Reg No: 95072452006)

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS
IN
FACULTY OF INFORMATION AND COMMUNICATION ENGINEERING

DEPARTMENT OF MCA

FRANCIS XAVIER ENGINEERING COLLEGE
(Autonomous)

TIRUNELVELI – 627 003

APRIL 2026

FRANCIS XAVIER ENGINEERING COLLEGE
(Autonomous)

BONAFIDE CERTIFICATE

Certified that this Report titled **"SMART FOOD SERVICE SYSTEM"** is the bonafide work of ARUN GLADSON S (Reg.No.95072452006) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Supervisor	Head of the Department
Dr M Vijaya Maheswari	Mrs. D. Angeline Ranjithamani
Assistant Professor	Assistant Professor & HOD
Department of MCA	Department of MCA
Francis Xavier Engineering College	Francis Xavier Engineering College
(Autonomous)	(Autonomous)
Tirunelveli – 627 003	Tirunelveli – 627 003

Submitted for the MCA Degree Project work Viva Voce held on …………………..

INTERNAL EXAMINER EXTERNAL EXAMINER

DECLARATION

I affirm that the project work titled “**SMART FOOD SERVICE SYSTEM**” being submitted in partial fulfillment for the award of Master of Computer Applications is the original work carried out by me. It has not formed the part of any other project work submitted for the award of any degree or diploma, either in this or any other University.

ARUN GLADSON S

95072452006

I certify that the declaration made above by the candidate is true

Signature of the Guide,
Dr M Vijaya Maheswari
Assistant Professor
Department of MCA
Francis Xavier Engineering College
(Autonomous)
Tirunelveli – 627 003

ACKNOWLEDGEMENT

I first thank the **Lord Almighty** who made me energetic and feel tenacious to complete this project in all success.

I express my sincere thanks to **Dr.S.Cletus Babu, Chairman** and **Mrs.J.X.Amali Cletus Babu, Vice Chairman, SCAD Group of institution** who always cared to fulfill need of the students and be a moral support to me.

I deeply thank **Dr. V. Velmurugan, ME.,Ph.D., Principal, Francis Xavier Engineering College** for having given me a chance to pursue my M.C.A Post Graduation programme in this estimated institution.

I am indeed to give my gratitude to **Mr. Zahariya Gabriel.,M.E.,(Ph.D), Campus Manager, Francis Xavier Engineering College,** for his moral support.

I whole heartily thank the Supervisor **Dr M Vijaya Maheswari** and HOD **Mrs.D.Angeline Ranjithamani MCA.,M.Phil.,M.E.(Ph.D), Assistant Professor & Head , Department of Computer Application, Francis Xavier Engineering College** for her valuable guidance and motivation.

I must be very grateful to all the **Staff** of our departments who have been the source of encouragement. I thank all my **non-teaching staff members** of my departments for their valuable advice and pleasing coordination throughout my project.

Thanks also to my friends and family who have backed me up and support aspiration with real encouragements and for their indispensible suggestions in each and every place of my project.

ABSTRACT

The Smart Food Service System is a web-based application that helps customers order food easily from a restaurant. Users can view the menu, choose food items, place orders, and make payments online. The system also allows customers to track their order status in real time.Admins can manage menu items, categories, orders, and users through an admin panel. This system reduces manual work, saves time, and improves accuracy in restaurant operations.The Smart Food Service System makes food ordering faster, easier, and more efficient for both customers and restaurant staff.

திட்டப்பணிச் சுருக்கம்

ஸ்மார்ட் உணவு சேவை அமைப்பு என்பது ஒரு இணைய அடிப்படையிலான பயன்பாடு ஆகும். இது வாடிக்கையாளர்கள் உணவகத்தில் இருந்து உணவை எளிதாக ஆர்டர் செய்ய உதவுகிறது.

பயனர்கள் மெனுவைப் பார்க்கலாம், உணவு பொருட்களைத் தேர்வு செய்யலாம், ஆர்டரை பதிவு செய்யலாம் மற்றும் ஆன்லைன் மூலம் கட்டணம் செலுத்தலாம். இந்த அமைப்பு மூலம் வாடிக்கையாளர்கள் தங்கள் ஆர்டர் நிலையை நேரடியாக (real-time) கண்காணிக்கவும் முடியும்.

நிர்வாகிகள் இந்த அமைப்பில் மெனு பொருட்கள், பிரிவுகள், ஆர்டர்கள் மற்றும் பயனர்களை நிர்வாகப் பகுதி (admin panel) மூலம் நிர்வகிக்கலாம். இந்த அமைப்பு கைமுறை பணியை குறைத்து, நேரத்தை மிச்சப்படுத்தி, உணவக செயல்பாடுகளில் துல்லியத்தை அதிகரிக்கிறது.

ஸ்மார்ட் உணவு சேவை அமைப்பு உணவு ஆர்டர் செய்வதை வாடிக்கையாளர்களுக்கும் உணவக பணியாளர்களுக்கும் வேகமாகவும், எளிதாகவும், திறமையாகவும் மாற்றுகிறது.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

TERMS	EXPANSION
API	Application Programming Interface
ASP.NET	Active Server Pages Network Enabled Technologies
CSS	Cascading Style Sheet
HTML	Hypertext Markup Language
JS	Javascript
SSMS	Sql Server Management Studio
SQL	Structured Query Language
SMTP	Simple Mail Transfer Protocol

CHAPTER 1

INTRODUCTION

1.1 COMPANY PROFILE

Workline is a unique Software as a Service (SaaS) Human Resource Management System (HRMS) designed to address the specific needs of various industries, including retail, manufacturing, and technology companies. It offers comprehensive HR modules, enterprise-grade security features, and powerful analytics, making it a versatile tool for driving business transformation.

At Workline, the team is described as passionate puzzlers, dedicated to solving the unique challenges of HR management. With decades of experience, they have encountered diverse organizational setups, including multi-country locations, extensive contracted vendors, and proprietary performance measurement systems. Workline prides itself on its ability to adapt its HRMS system to a wide range of possibilities and configurations, tailored to suit each client's specific structure.

Our logo reflects our commitment to flexibility and problem-solving. We thrive on tackling complex challenges and finding innovative solutions. When others say "impossible," we say "challenge accepted." Our team is determined to make your workspace efficient, systematic, and happy.

API provides a simple RESTful interface with lightweight JSON responses, making integration easy. It offers various endpoints for attendance, workforce info, and leave management.

1.1.1 Vision

Solving every single puzzle inside every single workspace for every single person, Globally

1.1.2 Mission

Solving every single puzzle inside every single workspace for every single person

1.2 OVERVIEW OF THE PROJECT

The Smart Food Service System is a modern web-based application designed to simplify the process of ordering food in restaurants. In today's fast-paced world, customers prefer quick and convenient services, and this system helps meet those expectations by providing an easy-to-use online platform. Through this system, customers can browse the menu, select their desired food items, place orders, and make payments online without any hassle. It also allows users to track their order status in real time, ensuring transparency and better user experience. On the other hand, the system provides an admin panel for restaurant staff to efficiently manage menu items, categories, customer orders, and user details. This reduces manual work, minimizes errors, and improves overall operational efficiency. Overall, the Smart Food Service System enhances the food ordering experience by making it faster, more accurate, and user-friendly for both customers and restaurant management.

1.2.1 APPLICATIONS

The Smart Food Service System can be applied in various food service environments where efficient ordering and management are required. It provides a centralized platform for handling food orders, menu management, and customer interactions smoothly. One major application is in restaurants, where the system can be used to manage online orders, display digital menus, and process payments efficiently. It helps customers place orders easily and reduces waiting time. The system can also be used in cafeterias such as those in colleges and offices, allowing users to order food in advance and avoid long queues during busy hours. Another important application is in hotels, where guests can order food from their rooms using the platform, improving convenience and service quality. It is also useful in fast food chains and food courts, where large volumes of orders need to be handled quickly and accurately. In addition, cloud kitchens and online food delivery services can use this system to manage orders, track deliveries, and streamline operations without a physical dining space. Overall, the Smart Food Service System supports modern food services by providing a fast, organized, and user-friendly solution for managing food orders and improving customer satisfaction.

1.2.3 CHARACTERISTICS

• **User-Friendly Interface** – The system provides a simple and easy-to-use interface that allows customers to browse the menu, select food items, and place orders without difficulty.

• **Scalability** – The system can handle a large number of users and orders at the same time, making it suitable for restaurants of different sizes.

• **Secure Authentication** – It includes secure login and user authentication to ensure that only authorized users and admins can access the system.

• **Efficient Order Management** – Administrators can easily manage orders, update their status, and ensure smooth processing of customer requests.

• **Accessibility** – As a web-based system, users can access it anytime and from any device with an internet connection.

• **Efficient Data Storage** – The system stores user details, menu items, orders, and payment information in a structured database for easy management and retrieval.

1.2.4 MODULES DESCRIPTION

• **User Module** – This module manages customer activities such as registration, login, and profile management. It allows users to create accounts, log in securely, and manage their personal details.

• **Menu Module** – This module is responsible for displaying and managing food items. Admins can add, update, or remove menu items, categories, and prices, while users can view and select items easily.

• **Order Module** – This module allows customers to place food orders by selecting items from the menu. It also maintains order details such as quantity, total cost, and order history.

• **Payment Module** – The payment module handles online transactions. It allows users to make secure payments using different methods and ensures smooth processing of payments.

• **Order Tracking Module** – This module helps customers track their order status in real time, such as order placed, preparing, and delivered, improving transparency and user experience.

.

CHAPTER 2

LITERATURE SURVEY

2.1 CHALLENGES IDENTIFIED

During the study of existing online food ordering systems and restaurant management platforms, several challenges have been identified that affect the efficiency and user experience of these systems. These challenges highlight the need for a smart and user-friendly food service system.

• **Lack of Organized Menu Structure** – Many systems do not present menu items in a clear and categorized way, making it difficult for customers to find their desired food items.

• **Difficulty in Item Selection** – Customers often struggle to quickly find suitable food items due to poor search and filtering options.

• **Limited Order Tracking** – Some systems do not provide real-time order tracking, causing confusion about order status.

• **Poor User Experience** – Complicated interfaces and slow navigation make it difficult for users to place orders.

• **Lack of Personalization** – Many systems do not suggest food items based on customer preferences or past orders.

• **Inefficient Order Management** – Restaurant staff may face difficulties in handling multiple orders, updating status, and managing menu items effectively.

2.2 LITERATURE REVIEW

K. P. Singh, et al. [1] studied the impact of online food ordering systems in improving customer convenience and service efficiency. The research explains how digital platforms allow users to place orders easily from anywhere and reduce waiting time.

A. Kumar, et al. [2] proposed a framework for developing web-based food service systems that support menu management and order processing. The study highlights the importance of user-friendly design and efficient system performance.

S. Johnson, et al. [3] focused on recommendation systems in food applications. The study explains how customer preferences and past orders can be used to suggest suitable food items, improving user satisfaction.

M. Chen, et al. [4] introduced a model for improving food item discovery using categorization and filtering techniques. The research shows how proper grouping of items helps users find food easily.

R. Patel, et al. [5] proposed a system for tracking order status in real time. The study highlights how tracking systems improve transparency and customer trust.

L. Wang, et al. [6] explored cloud-based food ordering systems that support a large number of users. The research emphasizes scalability and accessibility of web-based platforms.

T. Nguyen, et al. [7] studied user experience in online ordering systems and identified factors like simple interface and fast response time as key elements for customer satisfaction.

P. Sharma, et al. [8] discussed database-driven systems for managing food orders, menu details, and customer data efficiently using structured storage.

J. Lee, et al. [9] proposed an integrated system combining menu management, order processing, and payment modules to improve overall efficiency.

S. Reddy, et al. [10] focused on secure authentication and payment systems in online platforms, ensuring safe transactions and protection of user data.

.

2.3 CHALLENGE 1

One of the major challenges in food ordering systems is the lack of a structured menu organization. Many systems contain a large number of food items, but they are not properly categorized, making it difficult for users to find what they want. This can lead to confusion, increased ordering time, and poor user experience. To solve this issue, the system must provide a well-organized menu structure.

• Organizing Menu Items Properly: Food items should be categorized based on type such as starters, main course, beverages, etc., so that users can easily browse and select items.

• Improving Menu Navigation: The system should include features like categories, filters, and search options to help users quickly find their desired food items.

2.4 CHALLENGE 2

Another major challenge in food service systems is the difficulty in selecting suitable food items. Customers may find it hard to choose from a large menu without proper guidance or filtering options. This can lead to delays and reduced user satisfaction. Therefore, improving item discovery is very important.

• Improving Search and Filtering: The system should allow users to search food items based on category, price, or keywords, making the selection process faster and easier.

• Providing Recommendations: The system can suggest food items based on customer preferences or previous orders, helping users make better and quicker decisions.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In the existing system, most restaurants use manual methods for food ordering and management. Customers place orders directly with staff, and orders are recorded on paper or basic systems. This process is time-consuming and prone to errors such as wrong orders, billing mistakes, and delayed service. There is no centralized system to manage menu updates, track orders, or analyze sales data. Payment is often handled manually, which increases the chances of mistakes and inefficiency. Overall, the existing system lacks automation, speed, and accuracy

3.1.1 Disadvantages

- Manual order taking
- High chance of human errors
- Slow service
- No real-time order tracking
- Difficult to manage menu and orders

3.2 PROPOSED SYSTEM

The proposed system is a web-based Smart Food Service System that automates the food ordering and restaurant management process. Customers can view the menu, place orders, and make payments online using a digital platform. The system provides role-based access for admins and users. Admins can manage menu items, categories, orders, and users through an admin dashboard. The system supports real-time order tracking, secure authentication, and digital payments

3.2.1 Advantages

- Online food ordering system
- Fast and accurate order processing
- Real-time order tracking
- Secure login and payment
- Easy menu and order management
- Automated reports and analytic

3.3 FEASIBILITY STUDY

3.3.1 Objective Of Feasibility Study

The main objective of the feasibility study is to analyze whether the proposed system is technically, economically, and operationally possible to implement. It helps identify potential challenges and ensures that the system can be developed within the available resources, time, and cost while meeting the needs of users.

3.3.2 Feasibility Study Parts

The feasibility study will be divided into the following key parts:

- Economic Feasibility
- Technical Feasibility
- Operational Feasibility

3.3.3 Economic Feasibility

Economic feasibility evaluates the cost required to develop and implement the **Course Centric Learning Platform**. The system can be developed using available technologies such as React JS, ASP.NET, and SQL Server, which reduces development costs. Since the platform is software-based and does not require expensive hardware, the overall cost of development and maintenance is manageable.

3.3.4 Technical Feasibility

Technical feasibility examines whether the required technology and technical resources are available to develop the system. The proposed platform uses modern and widely supported technologies such as React JS for the frontend, ASP.NET for backend development, and SQL Server for database management. These technologies are reliable and capable of supporting the functionalities required for the system.

3.3.5 Operational Feasibility

Operational feasibility determines whether the system can be effectively used and maintained by users and administrators. The proposed system is designed with a user-friendly interface that allows students to easily browse courses, enroll, and track progress. Administrators can efficiently manage course content and user data, making the system practical for everyday use.

3.4 SOFTWARE REQUIREMENTS

Operating System: Windows 10 / Windows 11

Frontend: React.js

Backend: C# (ASP.NET)

Database: MySQL

Browser: Google Chrome, Mozilla Firefox, Microsoft Edge

Code Editor: Antigravity

Version Control Tool: GitHub

API Testing Tool: Postman or Thunder Client

3.5 HARDWARE REQUIREMENT

Processor: Intel i5 or above

RAM: 8 GB or more

Storage: Minimum 100 GB of available space

Network: Stable high-speed internet connection.

3.6 SOFTWARE DESCRIPTION

3.6.1 Front End

The frontend of the Smart Food Service System is developed using **React.js**, a modern JavaScript library used to build dynamic and interactive user interfaces. React.js allows the application to be divided into reusable components, making development easier and more efficient. Through the frontend, users can browse the menu, select food items, place orders, make payments, and track their order status. The interface is designed to be simple, responsive, and user-friendly, allowing access from desktops, tablets, and mobile devices. React.js uses a Virtual DOM, which improves performance by updating only necessary parts of the page, ensuring a smooth and fast user experience.

3.6.2 Back End

The backend of the Smart Food Service System is developed using **C# with ASP.NET**, which is a powerful framework for building secure and scalable web applications. The backend handles core functionalities such as user authentication, menu management, order processing, payment handling, and order tracking. It also provides APIs that connect the React frontend with the database. ASP.NET ensures secure data handling, proper validation of user inputs, and efficient processing of requests. It helps maintain smooth communication between the frontend and backend.

CHAPTER 4

SYSTEM DESIGN

4.1 PROCESS MODEL

4.1.1 ARCHITECTURE DIAGRAM

It provides a visual representation of the system's structure and components.

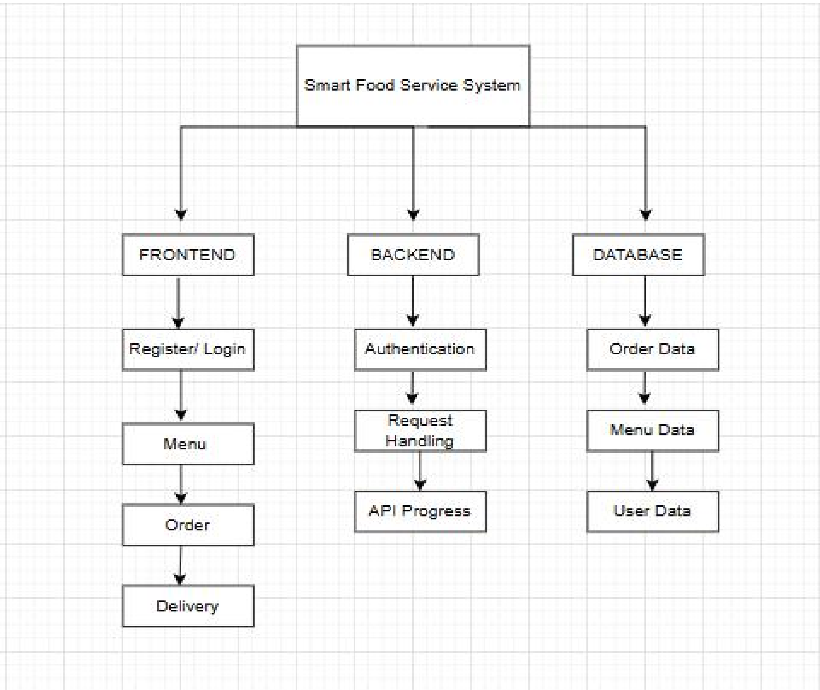

Fig 4.1 Architecture Diagram

4.1.2 USE CASE DIAGRAM

It illustrates the interactions between actors and the system's functionalities.

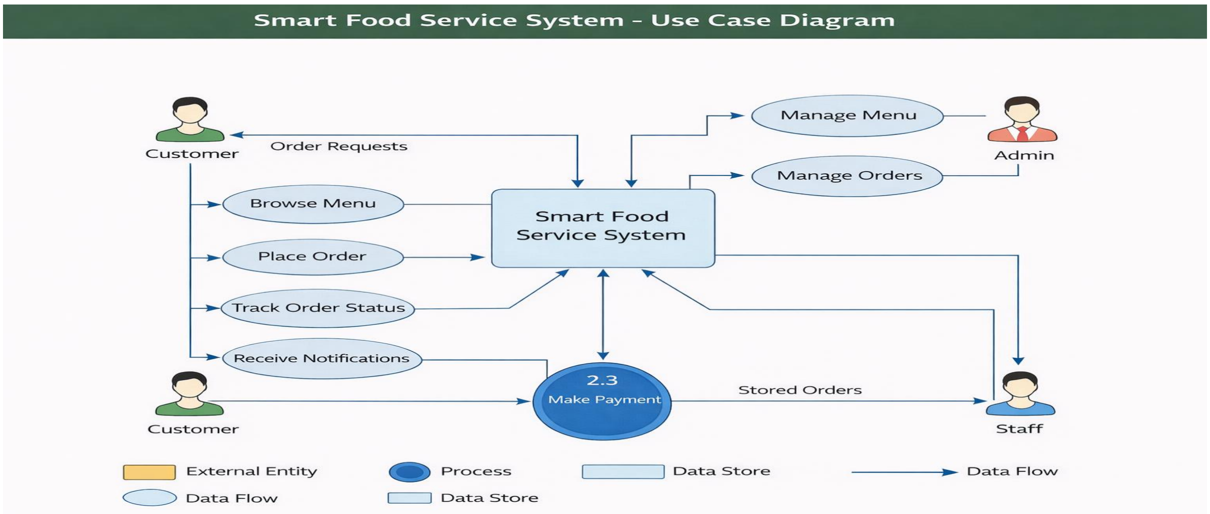

Fig 4.2 Use case Diagram

4.1.3 CLASS DIAGRAM

It presents the structure of the system by showing classes, attributes, and their relationships.

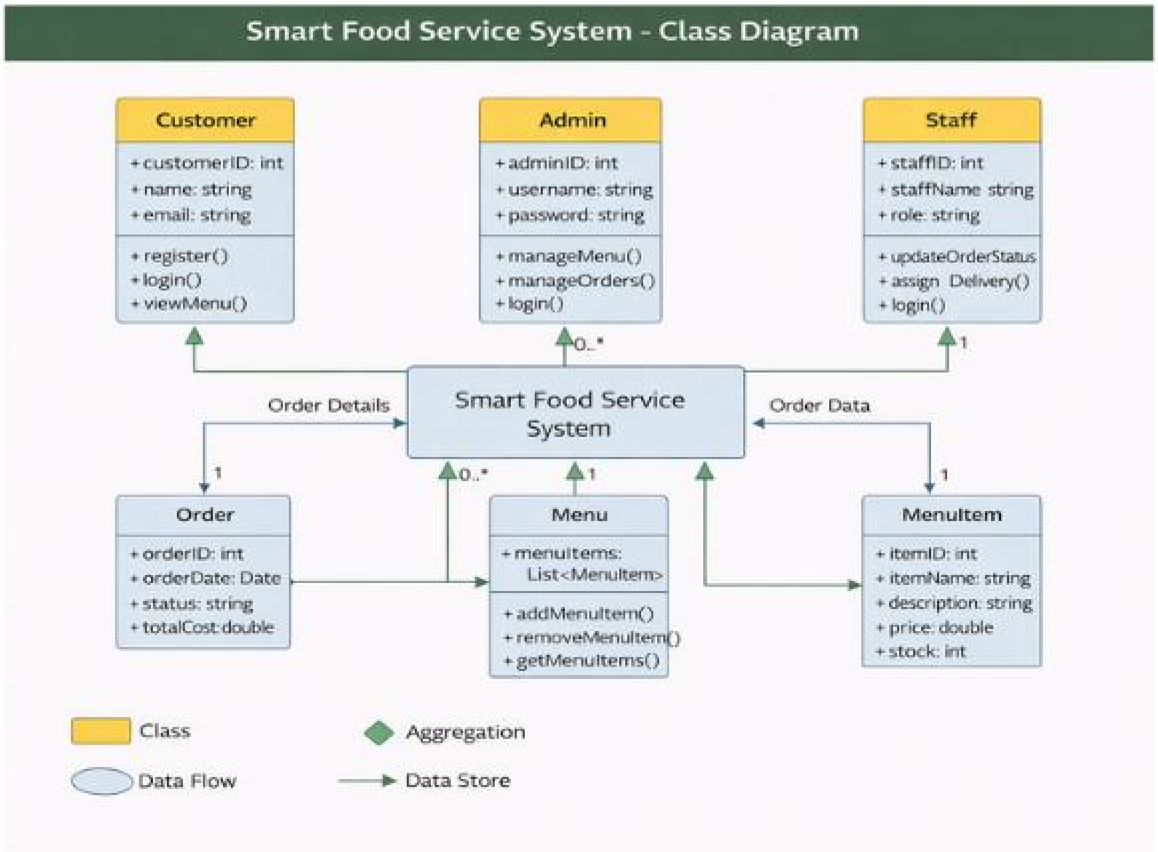

Fig 4.3 Class Diagram

4.1.4 DFD DIAGRAM

It represents the flow of data within a system, showing processes, data stores, and data flows.

DFD Level 0

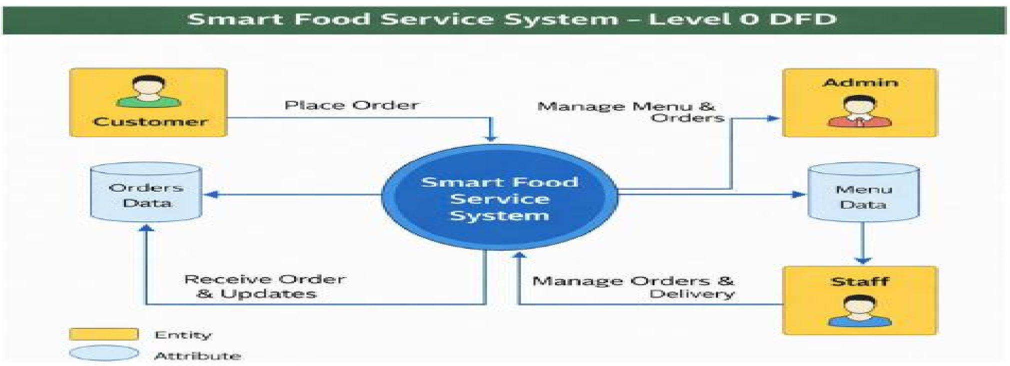

Fig 4.4 DFD Level 0

DFD Level 1

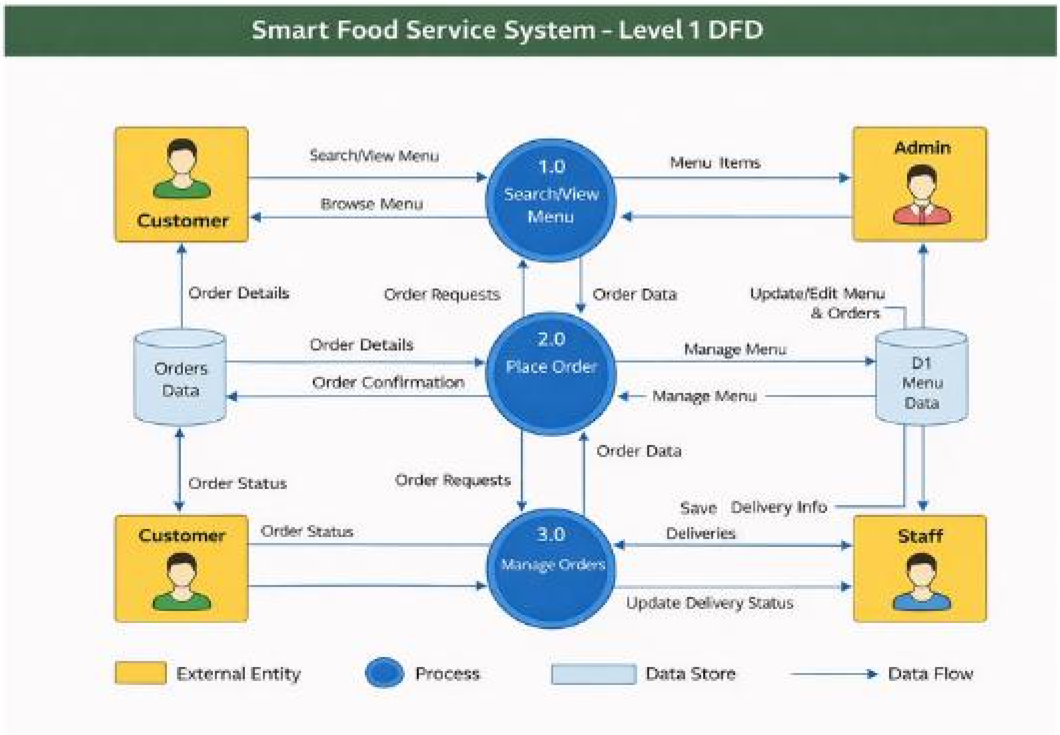

Fig 4.5 DFD Level 1

DFD Level 2

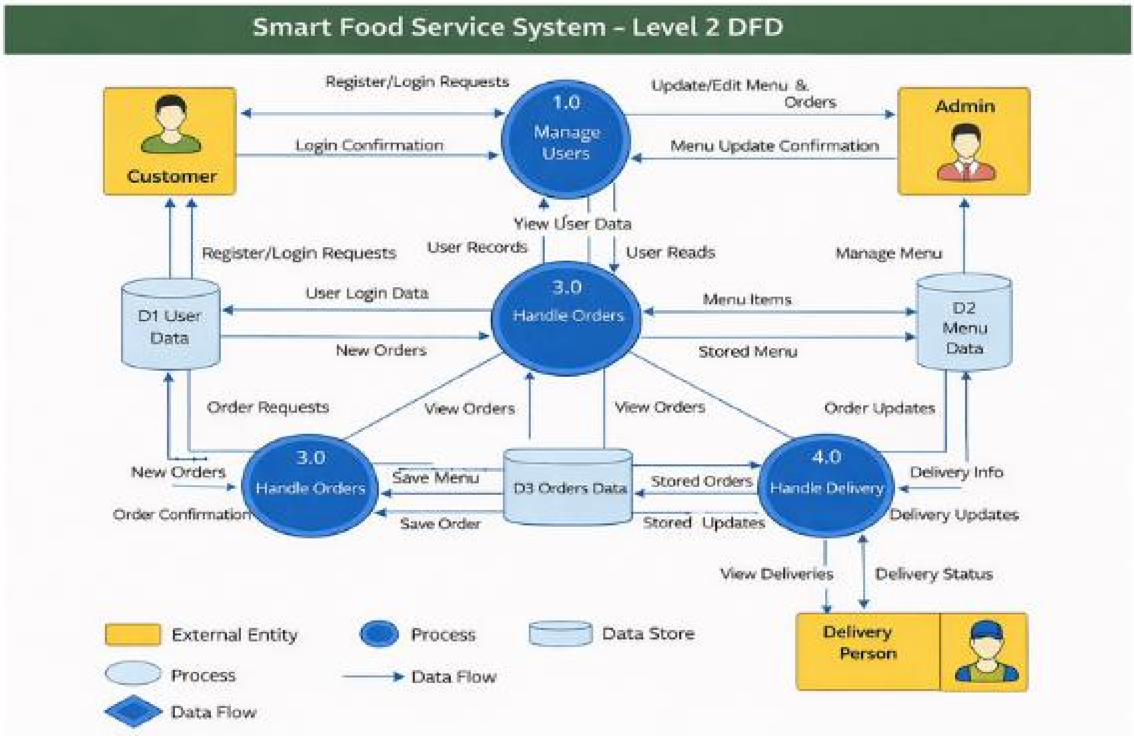

Fig 4.6 DFD Level 2

4.1.5 TABLE DESIGN

Database Name: SmartFoodServicedb

Table Name: users

Table 4.1 Users

Purpose: This table is used to maintain Register details

S. No	Field name	Datatype	Constraints
1	id	int	Primary key
2	name	varchar(50)	Not null
3	email	varchar(50)	Not null
4	password	varchar(50)	Not null
5	Role	varchar(50)	Not null
6	Phone	int	Not null
7	Address	varchar(50)	Not null

Table Name: Category

Table 4.2 Category

Purpose: This table is used to maintain products details

S. No	Field Name	Data Type	Description
1	Category_ID	int (PK)	Primary Key
2	Category_Name	varchar	Not Null

Table Name: Menu Item

Table 4..3 Menu Item

Purpose: This table is used to store the ordered items of the user

S. No	Field name	Datatype	Constraints
1	Menu_id	int	Primary key
2	Category_Name	Varchar	Foreign Key
3	Description	varchar	Not null
4	Price	int	Not null
5	Image_Url	Varchar	Not null

Table Name: Order

Table 4.4 Order Table

Purpose: This table is used to view the Order of the user

S. No	Field name	Datatype	Constraints
1	Customer_id	int	Primary key
2	Category_id	int	Foreign Key
3	Order_Date	int	Not Null
4	Total_Amount	int	Not null
5	Order_status	varchar	Not null
6	Payment_Status	Varchar	Not null

Table Name: Order Item

Table 4.5 Order Item Table

Purpose: This table is used to store the Order Item details shown by the Customer

S. No	Field Name	Data Type	Constraints
1	Order_item_id	int	Primary Key
2	Order_ID	int	Foreign Key
3	Category_id	int	Foreign Key
4	Quantity	int	Not NULL
5	Price	int	Not NULL

Table Name: Payment

Table 4.6 Payment Table

Purpose: This table is used to store the payment details

S. No	Field Name	Data Type	Constraints
1	Payment_ID	int	PRIMARY KEY
2	Order_ID	int	Foreign Key
3	Payment_Method	Varchar	Not NULL
4	Amount	int	Not NULL
5	Payment_Status	Varchar	Not NULL

Table Name: Delivery

Table 4.7 Delivery Table

Purpose: This table is used to track the delivery status shown by the Customer

S. No	Field Name	Data Type	Constraints
1	Delivery_ID	int	PRIMARY KEY
2	Order_ID	int	Foreign Key
3	Delivery_Staff_ID	int	Not NULL
4	Status	Varchar	Not NULL
5	Payment_Status	Varchar	Not NULL
6	Location	Varchar	Not NULL

4.1.6 DATA DICTIONARY

Table 4.8 Data Dictionary - Users

S. No	Field name	Datatype	Description
1	id	int	Store id
2	name	varchar(50)	Store name
3	email	varchar(50)	Store email
4	password	varchar(50)	Store password
5	Role	varchar(50)	Defines whether the user is Admin or Customer or Staff
6	Phone	int	Store the Customer phone for OTP verification
7	Address	varchar(50)	Store address for delivery order

Table 4.9 Data Dictionary - Category data

S. No	Field name	Datatype	Description
1	Category_ID	int (PK)	Store category id
2	Category_Name	varchar	Store the category name

Table 4.10 Data Dictionary - Menu Item data

S. No	Field name	Datatype	Description
1	Menu_id	int	Store menu id
2	Category_Name	Varchar	Store category name
3	Description	varchar	Store enrolled description

4	Price	int	Store enrollment price
5	Image_Url	Varchar	Store enrollment image url

Table 4.11 Data Dictionary - Order data

S. No	Field name	Datatype	Description
1	Customer_id	int	Store customer id
2	Category_id	int	Store related category id
3	Order_Date	int	Store order date
4	Total_Amount	int	Store total amount of order item
5	Order_status	varchar	Store order status
6	Payment_Status	varchar	Store payment status

Table 4.12 Data Dictionary - Order Item data

S. No	Field name	Datatype	Description
1	Order_item_id	int	Store order item id
2	Order_ID	int	Store related order id
3	Category_id	int	Store related category id
4	Quantity	int	Store quantity of order item
5	Price	int	Store order price

Table 4.13 Data Dictionary - Payment data

S. No	Field name	Datatype	Description
1	Payment_ID	int	Store payment id
2	Order_ID	int	Store related order id
3	Payment_Method	Varchar	Store payment type for order item
4	Amount	int	Store amount for order item
5	Payment_Status	Varchar	Store order payment status

Table 4.14 Data Dictionary - Delivery data

S. No	Field name	Datatype	Description
1	Delivery_ID	int	Store delivery id
2	Order_ID	int	Store related order id
3	Delivery_Staff_ID	int	Store related staff delivery order
4	Delivery_Status	Varchar	Store delivery item
5	Payment_Status	Varchar	Store payment status

4.1.9 ER DIAGRAM

It displays the entities, attributes, and relationships within a database system.

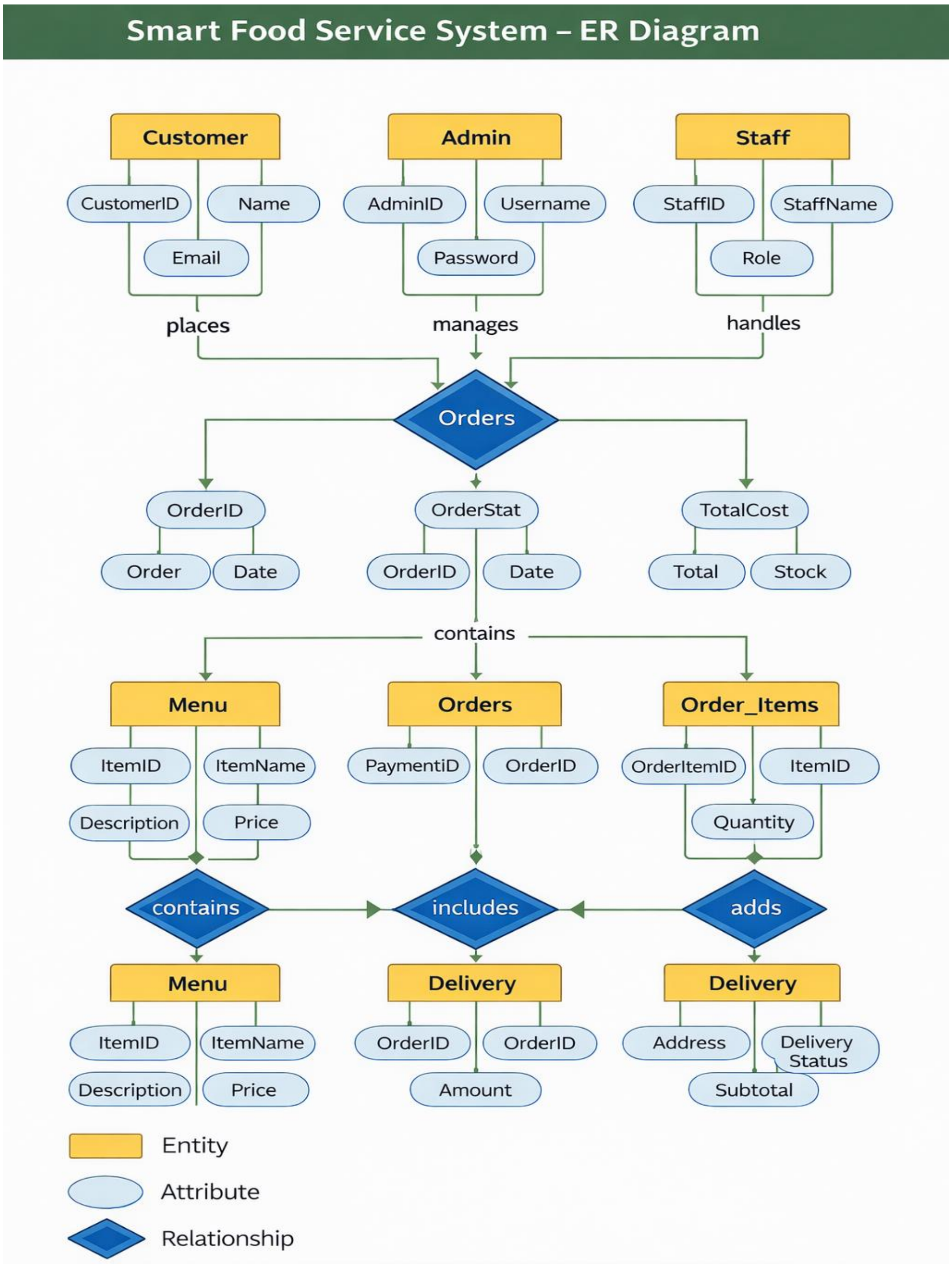

Fig 4.7 ER Diagram

4.2 SYSTEM IMPLEMENTATION

4.2.1 PROGRAM DESIGN LANGUAGE

User Registration Module

```
START
DISPLAY registration form
INPUT name, email, password, role
CHECK whether email already exists in USERS table
IF email exists THEN
    DISPLAY "User already registered"
ELSE
    INSERT user details into USERS table
    DISPLAY "Registration successful"
END IF
STOP
```

User Login Module

```
START
DISPLAY login form
INPUT email and password
SEARCH USERS table for matching email and password
IF match found THEN
    CHECK user role
    IF role = Admin THEN
        REDIRECT to Admin Dashboard
    ELSE
        REDIRECT to Student Dashboard
    END IF
ELSE
    DISPLAY "Invalid email or password"
END IF
STOP
```

Menu Management Module (Admin)

```
Display options:
  START
ADMIN selects option (Add / Update / Delete Menu)
IF Add Menu THEN
    INPUT menu details
    INSERT into MENU table
    DISPLAY "Menu added successfully"
ELSE IF Update Menu THEN
    SELECT menu_id
    MODIFY menu details
    UPDATE Menu table
```

```
    DISPLAY "Menu updated successfully"
ELSE IF Delete Menu THEN
    SELECT Menu_id
    DELETE from Menu table
    DISPLAY "Menu deleted successfully"
END IF
STOP
```

Payment Track Module

```
START
STUDENT selects payment
CHECK if customer already enrolled in selected payment
IF already enrolled THEN
    DISPLAY "Payment Successfully"
ELSE
    INSERT payment details into payment method table
    DISPLAY "payment successful"
END IF
STOP
```

4.3 SYSTEM TESTING

4.3.1 UNIT TESTING

Unit testing involves testing individual components or modules of the system independently to ensure that each part works correctly. In the Smart Food Service System, unit testing is performed on modules such as user login, menu management, order processing, and payment handling. Each function is tested with different inputs to verify correct outputs. This helps in identifying bugs at an early stage and ensures that each module performs as expected before integration

4.3.2 INTEGRATION TESTING

Integration testing is carried out to verify the interaction between different modules of the system. After unit testing, individual components are combined and tested as a group. In this system, integration testing ensures that modules such as the frontend interface, backend API, and database work together seamlessly. For example, it checks whether the order placed by a user is correctly stored in the database and reflected in the admin panel. This testing helps in identifying issues related to data flow and communication between modules.

4.3.3 VALIDATION TESTING

In this system, validation testing is performed by providing different inputs and verifying whether the outputs are correct and meaningful. For example, when a user enters valid login details, the system should successfully log in, whereas invalid details should display an error message. Similarly, while placing an order, the system must validate item selection, quantity, and payment details before confirming the order.

4.3.4 ACCEPTANCE TESTING

Acceptance testing is the final phase of testing where the system is evaluated to ensure it meets user requirements and expectations. It is usually performed by end-users or clients. In the Smart Food Service System, acceptance testing ensures that customers can easily browse the menu, place orders, and track them without any issues, while admins can efficiently manage the system. If the system meets all requirements and works as expected, it is approved for deployment.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 EXPERIMENTAL SETUP

The setup involves configuring technologies required for developing the Smart Food Order System. The system is designed to provide an efficient and user-friendly environment for managing online courses and learning activities. The setup includes:

- Installing **Microsoft Visual Studio** and configuring the **ASP.NET framework** for backend development.
- Designing and implementing the **database using SQL Server** to store user information, course details, enrollments, and assessment results.
- Developing responsive user interfaces using **React JS, HTML, CSS, and JavaScript** for an interactive learning experience.
- Integrating features such as **course management, enrollment functionality, and assessment modules**.
- Implementing **secure APIs and authentication mechanisms** to ensure safe communication between the frontend and backend systems.

5.2 APPENDIX 1-SAMPLE SOURCE CODE

- **Home.jsx**

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import {
  FiSearch, FiMapPin, FiShoppingCart, FiUser, FiChevronRight,
  FiStar, FiClock, FiArrowRight, FiFacebook, FiTwitter,
  FiInstagram, FiPlay, FiSmartphone, FiCheckCircle, FiTruck, FiCreditCard, FiActivity,
FiImage,
  FiArrowLeft
} from 'react-icons/fi';
import { useDispatch, useSelector } from 'react-redux';
import { fetchMenu } from '../../configure/order/menuSlice.js';
import { fetchCategory } from '../../configure/category/categorySlice.js';
import "../../style/user/Homepage.css";
```

```
const RESTAURANTS = [
  { id: 1, name: 'Pizza Hut', rating: 4.2, time: '25-30 mins', cuisine: 'Pizza, Fast Food', image: '/images/restaurant_food_grid.png', badge: 'Top Rated' },
  { id: 2, name: 'Burger King', rating: 4.5, time: '20-25 mins', cuisine: 'Burgers, American', image: '/images/restaurant_food_grid.png', badge: 'Fast Delivery' },
  { id: 3, name: 'Paradise Biryani', rating: 4.8, time: '35-40 mins', cuisine: 'Biryani, Indian', image: '/images/restaurant_food_grid.png', badge: 'Must Try' },
  { id: 4, name: 'Wow! Momo', rating: 4.1, time: '15-20 mins', cuisine: 'Chinese, Snacks', image: '/images/restaurant_food_grid.png', badge: '' },
  { id: 5, name: 'Starbucks', rating: 4.6, time: '10-15 mins', cuisine: 'Coffee, Desserts', image: '/images/restaurant_food_grid.png', badge: '' },
  { id: 6, name: 'Subway', rating: 4.3, time: '20-25 mins', cuisine: 'Healthy, Salads', image: '/images/restaurant_food_grid.png', badge: 'Healthy' },
];
const OFFERS = [
  { id: 1, title: '50% OFF', desc: 'Up to $10 on your first order', color: '#FF5200' },
  { id: 2, title: 'BOGO', desc: 'Buy 1 Get 1 Free on all Pizzas', color: '#4CAF50' },
  { id: 3, title: 'Free Delivery', desc: 'No delivery fee on orders above $20', color: '#2196F3' },
  { id: 4, title: 'Weekend Special', desc: 'Extra 20% cashback using PayTM', color: '#9C27B0' },
];
const TRENDING = [
  { id: 1, name: 'Grilled Chicken', price: 12.99, image: '/images/trending_dishes.png' },
  { id: 2, name: 'Salmon Poke Bowl', price: 15.50, image: '/images/trending_dishes.png' },
  { id: 3, name: 'Chocolate Lava', price: 6.99, image: '/images/trending_dishes.png' },
  { id: 4, name: 'Mango Smoothie', price: 5.50, image: '/images/trending_dishes.png' },
];
const TESTIMONIALS = [
  { id: 1, name: 'John Doe', text: 'The delivery was super fast and the food was still hot when it arrived. Amazing service!', rating: 5, img: 'https://i.pravatar.cc/150?img=1' },
  { id: 2, name: 'Sarah Smith', text: 'I love the variety of restaurants available. Ordering is so easy and the app is very smooth.', rating: 4, img: 'https://i.pravatar.cc/150?img=5' },
  { id: 3, name: 'Mike Johnson', text: 'Best food delivery app I have ever used. The live tracking is very accurate.', rating: 5, img: 'https://i.pravatar.cc/150?img=11' },
```

```
];
export default function Homepage() {
  const dispatch = useDispatch();
  const scrollRef = React.useRef(null);
  const { items: menuItems, loading: menuLoading } = useSelector(state => state.menu);
  const { items: categories, loading: categoryLoading } = useSelector(state => state.category);
  const [scrolled, setScrolled] = useState(false);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    dispatch(fetchMenu());
    dispatch(fetchCategory());
    const handleScroll = () => {
      setScrolled(window.scrollY > 50);
    };
    window.addEventListener('scroll', handleScroll);
    setTimeout(() => setLoading(false), 800);
    return () => window.removeEventListener('scroll', handleScroll);
  }, [dispatch]);
  const scroll = (direction) => {
    if (scrollRef.current) {
      const { scrollLeft, clientWidth } = scrollRef.current;
      const scrollTo = direction === 'left' ? scrollLeft - clientWidth * 0.7 : scrollLeft +
clientWidth * 0.7;
      scrollRef.current.scrollTo({ left: scrollTo, behavior: 'smooth' });
    }
  };
  const dynamicCategories = React.useMemo(() => {
    if (!categories || categories.length === 0) return [];
    return categories.map(cat => {
      // Find the first menu item in this category that has an image
      const itemWithImage = menuItems?.find(item =>
        item.category?.toLowerCase() === cat.name?.toLowerCase() && item.imageUrl
      );
```

```
    return {
      id: cat.id,
      name: cat.name,
      image: itemWithImage?.imageUrl || null,
      icon: CATEGORY_ICONS[cat.name] || '   ' // Fallback icon
    };
  });
}, [categories, menuItems]);

return (
  <div className="home-wrapper">
    <nav className={`navbar ${scrolled ? 'scrolled' : ''}`}>
      <Link to="/" className="nav-brand">
        <FiActivity className="logo-icon" />
        <span className="logo-text">SFSS</span>
      </Link>
      <div className="nav-search-container">
        <div className="search-bar">
          <FiSearch size={18} color="#9ca3af" />
          <input type="text" placeholder="Search for restaurant, cuisine or a dish..." />
        </div>
      </div>
      <div className="location-selector">
        <FiMapPin color="#FF5200" />
        <span>New York, USA</span>
      </div>
      <ul className="nav-links">
        <li><Link to="/" className="nav-link">Home</Link></li>
        <li><Link to="/restaurants" className="nav-link">Restaurants</Link></li>
        <li><Link to="/offers" className="nav-link">Offers</Link></li>
        <li className="cart-btn">
          <Link to="/cart" className="nav-link"><FiShoppingCart size={20} /><span
        className="cart-count">3</span></Link>
        </li>
```

```
    <li><Link to="/profile" className="nav-link"><FiUser size={20} /></Link></li>
  </ul>
</nav>
    <section className="hero">
  <div className="hero-content">
    <h1 className="animate-up">Delicious food, delivered to your doorstep</h1>
    <p className="animate-up" style={{ animationDelay: '0.2s' }}>
      Experience the taste of luxury from the comfort of your home.
      Freshly prepared, carefully packed, and delivered with love.
    </p>
    <div className="hero-btns animate-up" style={{ animationDelay: '0.4s' }}>
      <Link to="/menu" className="btn-primary">Order Now <FiArrowRight /></Link>
      <Link to="/restaurants" className="btn-secondary">Explore Restaurants</Link>
    </div>
  </div>
</section>

    <section className="section" style={{ paddingBottom: '40px' }}>
  <div className="section-header">
    <div>
      <h2 className="section-title">What's on your mind?</h2>
    </div>
    <div className="category-scroll-btns">
      <button className="scroll-btn" onClick={() => scroll('left')}>
        <FiArrowLeft />
      </button>
      <button className="scroll-btn" onClick={() => scroll('right')}>
        <FiArrowRight />
      </button>
    </div>
  </div>
  {categoryLoading || menuLoading ? (
    <div className="category-list skeleton-list">
      {Array(8).fill(0).map((_, i) => (
```

```
      <div key={i} className="category-item skeleton" style={{ width: '150px', height:
    '150px', borderRadius: '50%' }}></div>
    ))}
  </div>
) : dynamicCategories.length === 0 ? (
  <div style={{ textAlign: 'center', padding: '40px', color: '#9ca3af' }}>
    <FiImage size={40} style={{ marginBottom: '10px' }} />
    <p>No categories found yet.</p>
  </div>
) : (
  <div className="category-list" ref={scrollRef}>
    {dynamicCategories.map((cat, i) => (
      <div        key={cat.id        ||        i}        className="category-item        animate-up"
      style={{ animationDelay: `${i * 0.1}s` }}>
        <div className="category-icon-wrapper">
          {cat.image ? (
            <img
              src={cat.image}
              alt={cat.name}
            />
          ) : (
            cat.icon
          )}
        </div>
        <p>{cat.name}</p>
      </div>
    ))}
  </div>
)}
<div className="section-divider"></div>
</section>
<section className="section" style={{ backgroundColor: 'var(--bg-light)' }}>
  <div className="section-header">
    <div>
```

```
          <h2 className="section-title">Exclusive Offers</h2>
          <p className="section-subtitle">Grab the best deals of the day</p>
        </div>
      </div>
    <div className="offers-container">
        <div className="offers-list">
          {OFFERS.map(offer => (
            <div key={offer.id} className="offer-card">
              <div className="offer-bg"></div>
              <div className="offer-content">
                <h3 className="offer-title">{offer.title}</h3>
                <p className="offer-desc">{offer.desc}</p>
                <button className="btn-secondary" style={{ marginTop: '20px', padding: '10px
                  20px', fontSize: '13px' }}>
                  Redeem Code
                </button>
              </div>
            </div>
          ))}
        </div>
      </div>
    </section>
<section className="section">
      <div className="section-header">
        <div>
          <h2 className="section-title">Top Rated Restaurants</h2>
          <p className="section-subtitle">The most loved places in your area</p>
        </div>
        <Link to="/restaurants" className="nav-link" style={{ fontSize: '14px' }}>
          View More <FiChevronRight />
        </Link>
      </div>

      <div className="restaurant-grid">
```

```
{loading ? (
  Array(6).fill(0).map((_, i) => (
    <div key={i} className="restaurant-card skeleton" style={{ height:
    '320px' }}></div>
  ))
) : (
  RESTAURANTS.map(res => (
    <div key={res.id} className="restaurant-card">
      <div className="res-img-wrapper">
        {res.badge && <span className="res-badge">{res.badge}</span>}
        <img src={res.image} alt={res.name} />
      </div>
      <div className="res-info">
        <div className="res-name-row">
          <h3 className="res-name">{res.name}</h3>
          <div className="res-rating"><FiStar size={12} fill="white" />
              {res.rating}</div>
        </div>
        <p style={{ color: 'var(--text-light)', fontSize: '14px' }}>{res.cuisine}</p>
        <div className="res-details">
          <div className="res-detail-item"><FiClock size={16} color="var(--primary)"
            /> {res.time}</div>
          <div className="res-detail-item"><strong>$$</strong></div>
        </div>
      </div>
    </div>
  ))
)}
</div>
</section>
<section className="section" style={{ backgroundColor: 'var(--bg-light)' }}>
<div className="section-header">
<div>
  <h2 className="section-title">Trending Dishes</h2>
```

```
      <p className="section-subtitle">People are loving these right now</p>
    </div>
  </div>
  <div className="restaurant-grid" style={{ gridTemplateColumns: 'repeat(auto-fill,
      minmax(260px, 1fr))' }}>
    {TRENDING.map(dish => (
      <div key={dish.id} className="dish-card">
        <img src={dish.image} alt={dish.name} className="dish-img" />
        <div className="dish-info">
          <h4>{dish.name}</h4>
          <div className="rating-stars">
            <FiStar fill="#ffb400" size={14} /><FiStar fill="#ffb400" size={14} /><FiStar
          fill="#ffb400" size={14} /><FiStar fill="#ffb400" size={14} /><FiStar
          fill="#ffb400" size={14} />
          </div>
          <div className="dish-price-row">
            <span className="dish-price">${dish.price}</span>
            <button className="btn-add">Add to Cart</button>
          </div>
        </div>
      </div>
    ))}
  </div>
</section>

  <section className="section">
  <div className="features-grid">
    <div className="feature-card">
      <FiTruck className="feature-icon" />
      <h3>Fast Delivery</h3>
      <p>Your food will be at your door within 30 minutes, guaranteed.</p>
    </div>
    <div className="feature-card">
      <FiActivity className="feature-icon" />
```

```
      <h3>Order Tracking</h3>
      <p>Track your food in real-time from the kitchen to your doorstep.</p>
    </div>
    <div className="feature-card">
      <FiCreditCard className="feature-icon" />
      <h3>Secure Payments</h3>
      <p>Multiple payment options from credit cards to digital wallets.</p>
    </div>
  </div>
</section>
<section className="section" style={{ background: 'var(--bg-light)' }}>
  <h2 className="section-title" style={{ textAlign: 'center', marginBottom:
    '50px' }}>What our foodies say</h2>
  <div className="testimonial-container">
    {TESTIMONIALS.map(t => (
      <div key={t.id} className="testimonial-card">
        <div className="user-row">
          <img src={t.img} alt={t.name} className="user-img" />
          <div className="user-name">{t.name}</div>
        </div>
        <div className="rating-stars">
          {Array(t.rating).fill(0).map((_, i) => <FiStar key={i} fill="#ffb400" />)}
        </div>
        <p className="testimonial-text">"{t.text}"</p>
      </div>
    ))}
  </div>
</section>
<footer className="footer">
  <div className="footer-grid">
    <div className="footer-col">
      <Link to="/" className="footer-logo">SFSS</Link>
      <p className="footer-desc">
        Your favorite food delivered fast. We are committed to bringing you
```

```
    the best dining experience at home with the widest variety of restaurants.
  </p>
  <div className="social-links">
    <a href="#" className="social-link"><FiFacebook /></a>
    <a href="#" className="social-link"><FiTwitter /></a>
    <a href="#" className="social-link"><FiInstagram /></a>
  </div>
</div>
<div className="footer-col">
  <h4>Company</h4>
  <ul className="footer-links">
    <li><Link to="/about">About Us</Link></li>
    <li><Link to="/team">Our Team</Link></li>
    <li><Link to="/careers">Careers</Link></li>
    <li><Link to="/blog">Blog</Link></li>
  </ul>
</div>
<div className="footer-col">
  <h4>Support</h4>
  <ul className="footer-links">
    <li><Link to="/help">Help Center</Link></li>
    <li><Link to="/contact">Contact Us</Link></li>
    <li><Link to="/privacy">Privacy Policy</Link></li>
    <li><Link to="/terms">Terms of Service</Link></li>
  </ul>
</div>
<div className="footer-col">
  <h4>Experience SFSS App</h4>
  <div className="app-btns">
    <a href="#" className="app-btn">
      <FiPlay size={24} />
      <div><span>GET IT ON</span><strong>Google Play</strong></div>
    </a>
    <a href="#" className="app-btn">
```

```
            <FiSmartphone size={24} />
            <div><span>Download on the</span><strong>App Store</strong></div>
          </a>
        </div>
      </div>
    </div>
    <div className="footer-bottom">
      <p>© 2026 Smart Food Service System. All rights reserved.</p>
      <div style={{ display: 'flex', gap: '30px' }}>
        <span>Privacy</span>
        <span>Terms</span>
        <span>Cookies</span>
      </div>
    </div>
  </footer>
</div>
);
}
```

- **Category_manager.jsx**

```
import React, { useState } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { FiX, FiPlus, FiEdit2, FiTrash2, FiCheck, FiCornerDownRight } from 'react-icons/fi';
import { addCategory, updateCategory, deleteCategory } from
'../../../../configure/category/categorySlice.js';

const CategoryManager = ({ onClose }) => {
  const dispatch = useDispatch();
  const { items: categories, loading } = useSelector(state => state.category);

  const [newCategoryName, setNewCategoryName] = useState('');
  const [editingId, setEditingId] = useState(null);
  const [editName, setEditName] = useState('');
  const [error, setError] = useState('');
```

```
  const handleAdd = async (e) => {
    e.preventDefault();
    if (!newCategoryName.trim()) return;
    try {
      await dispatch(addCategory({ name: newCategoryName.trim() })).unwrap();
      setNewCategoryName('');
      setError('');
    } catch (err) {
      setError(err || 'Failed to add category');
    }
  };
  const handleUpdate = async (id) => {
    if (!editName.trim()) return;
    try {
      await dispatch(updateCategory({ id, name: editName.trim() })).unwrap();
      setEditingId(null);
      setEditName('');
      setError('');
    } catch (err) {
      setError(err || 'Failed to update category');
    }
  };
  const handleDelete = async (id) => {
    if (window.confirm('Are you sure you want to delete this category? Items in this category
might become uncategorized.')) {
      try {
        await dispatch(deleteCategory(id)).unwrap();
        setError('');
      } catch (err) {
        setError(err || 'Failed to delete category');
      }
    }
  };
  return (
```

```
<div className="mm-modal-overlay" onClick={onClose} style={{ zIndex: 1100 }}>
  <div className="mm-modal" onClick={e => e.stopPropagation()} style={{ maxWidth:
'450px' }}>
    <div className="mm-modal-header">
      <h2 className="mm-modal-title">Manage Categories</h2>
      <button className="mm-modal-close" onClick={onClose}>
        <FiX />
      </button>
    </div>

    <div className="mm-form" style={{ paddingBottom: '10px' }}>
      {error && <div style={{ color: '#ef4444', fontSize: '13px', marginBottom:
'10px' }}>{error}</div>}

      {/* Add New Category */}
      <form onSubmit={handleAdd} style={{ display: 'flex', gap: '8px', marginBottom:
        '16px' }}>
        <input
          type="text"
          className="mm-form-input"
          placeholder="New category name..."
          value={newCategoryName}
          onChange={(e) => setNewCategoryName(e.target.value)}
          disabled={loading}
        />
        <button type="submit" className="mm-btn-primary" style={{ padding: '10px',
        minWidth: '44px' }} disabled={loading}>
          <FiPlus />
        </button>
      </form>
      <div style={{ maxHeight: '300px', overflowY: 'auto', border: '1px solid #e5e7eb',
        borderRadius: '12px' }}>
        {categories.length === 0 ? (
```

```
<p style={{ padding: '20px', textAlign: 'center', color: '#9ca3af', fontSize: '14px' }}>No categories found.</p>
) : (
  <ul style={{ listStyle: 'none', padding: 0, margin: 0 }}>
    {categories.map((cat) => (
      <li key={cat.id} style={{
        padding: '12px 16px',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'space-between',
        borderBottom: '1px solid #f3f4f6'
      }}>
        {editingId === cat.id ? (
          <div style={{ display: 'flex', gap: '8px', width: '100%' }}>
            <input
              type="text"
              className="mm-form-input"
              value={editName}
              onChange={(e) => setEditName(e.target.value)}
              autoFocus
            />
            <button
              className="mm-btn-primary"
              style={{ padding: '6px 12px', background: '#0bb34c' }}
              onClick={() => handleUpdate(cat.id)}
            >
              <FiCheck />
            </button>
            <button
              className="mm-btn-secondary"
              style={{ padding: '6px 12px' }}
              onClick={() => setEditingId(null)}
            >
```

```
          </button>
        </div>
      ) : (
        <>
          <span style={{ fontSize: '14px', fontWeight: '500', color:
          '#374151' }}>{cat.name}</span>
          <div style={{ display: 'flex', gap: '4px' }}>
            <button
              className="mm-btn-edit"
              style={{ padding: '6px' }}
              onClick={() => {
                setEditingId(cat.id);
                setEditName(cat.name);
              }}
            >
              <FiEdit2 size={14} />
            </button>
            <button
              className="mm-btn-danger"
              style={{ padding: '6px' }}
              onClick={() => handleDelete(cat.id)}
            >
              <FiTrash2 size={14} />
          </div>
        </>
      )}
    </li>
  ))}
export default CategoryManage
```

5.3 APPENDIX 2-SAMPLE SCREENS

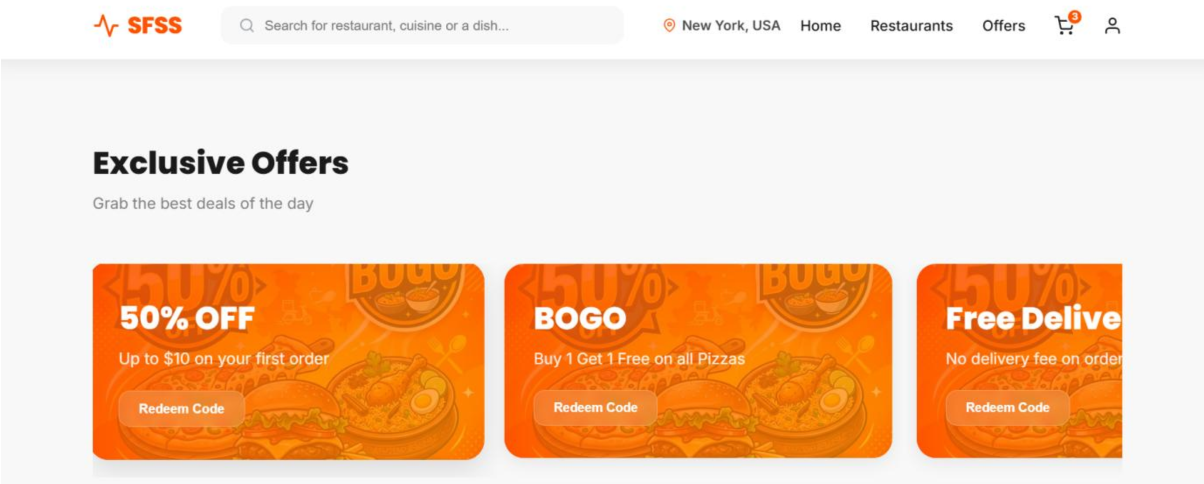

Fig A2.1. Home Screen

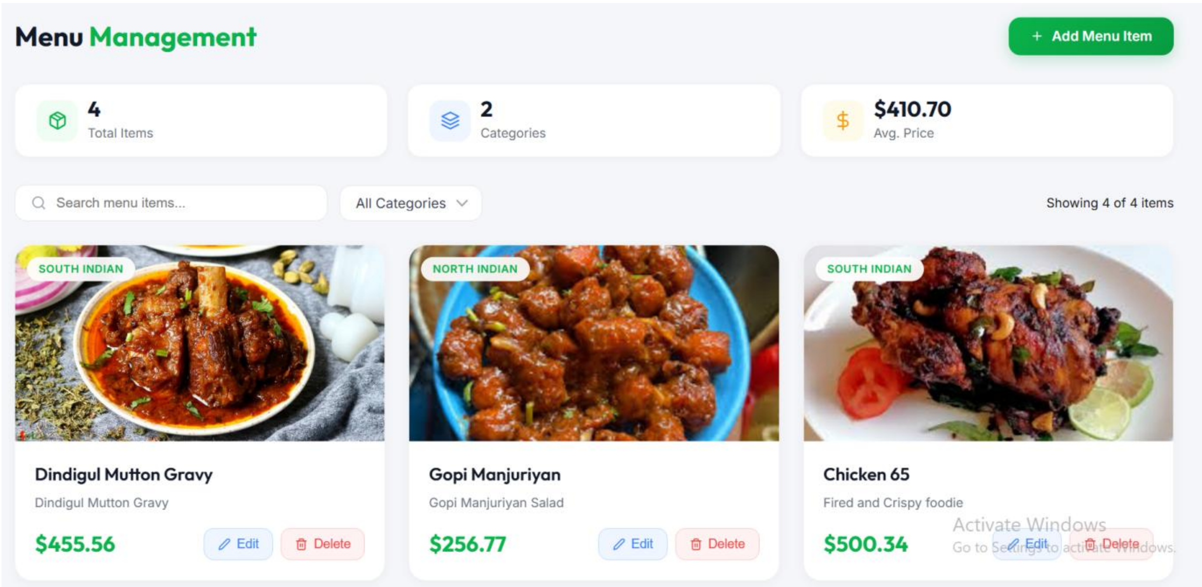

Fig A2.2. Menu Item

CHAPTER 6

CONCLUSION & FUTURE ENHANCEMENTS

6.1 CONCLUSION

The **Smart Food Service System** successfully provides a digital solution to simplify and enhance the food ordering process in restaurants. The system allows customers to conveniently browse menus, place orders, and make payments online, reducing waiting time and improving overall user experience from the administrative perspective, the system offers efficient management of menu items, categories, users, and orders through a centralized platform. This reduces manual effort, minimizes errors, and increases operational efficiency. The integration of frontend and backend technologies ensures smooth data flow, reliability, and system performance

.

6.2 FUTURE ENHANCEMENTS

● Develop an Android/iOS mobile app to provide better accessibility and user experience.

● Implement intelligent suggestions based on user preferences and order history.

● Enhance tracking by integrating live location features for delivery services.

● Add support for multiple payment gateways such as UPI, credit/debit cards, and digital wallets.

● Notify users about order status, offers, and discounts in real time.

CHAPTER 7

REFERENCES

7.1 SCOPUS REFERRED

[1] K. P. Singh, "Impact of Online Food Ordering Systems on Customer Satisfaction," *International Journal of Information Management*, vol. 45, pp. 123–130, 2019

[2] A. Kumar, "Design and Development of Web-Based Food Service Applications," *Journal of Web Engineering*, vol. 18, no. 2, pp. 95–110, 2020

[3] S. Johnson, "Recommendation Systems in Food Delivery Applications Using Machine Learning," *Expert Systems with Applications*, vol. 149, pp. 113–120, 2020..

[4] M. Chen, "Efficient Food Item Categorization and Search Optimization in Online Platforms," *Future Generation Computer Systems*, vol. 102, pp. 470–480, 2020

[5] R. Patel, "Real-Time Order Tracking System Using Web Technologies," *IEEE Access*, vol. 8, pp. 150000–150010, 2020.

[6] L. Wang, "Cloud-Based Scalable Architecture for Online Food Ordering Systems," *Journal of Cloud Computing*, vol. 9, no. 1, pp. 1–12, 2020.

[7] T. Nguyen, "User Experience Evaluation in Online Food Ordering Applications," *Computers in Human Behavior*, vol. 108, pp. 106–115, 2020.

[8] P. Sharma, "Database Design for Efficient Food Ordering and Management Systems," *Procedia Computer Science*, vol. 171, pp. 1345–1352, 2020.

[9] J. Lee, "Integrated Web-Based Food Ordering System with Payment Gateway," *IEEE Transactions on Engineering Management*, vol. 67, no. 3, pp. 845–856, 2020..

[10] S. Reddy, "Secure Authentication and Payment Systems in E-Commerce Applications," *Journal of Information Security and Applications*, vol. 52, pp. 102–110, 2020

7.2 WEBSITES REFERRED

- React JS Official Documentation: https://react.dev/learn
- ASP.NET Core Documentation: https://learn.microsoft.com/en-us/aspnet/core
- Microsoft SQL Server Documentation https://learn.microsoft.com/en-us/sql/sql-server
- W3Schools Web Sites: https://www.swiggy.com
- MDN Web Docs for JavaScript and Web Technologies: https://www.geeksforgeeks.org/

7.3 BOOKS REFERRED

- *Learning React: Modern Patterns for Developing React Apps"* by Alex Banks and Eve Porcello
- *"ASP.NET Core in Action"* by Andrew Lock
- *"Pro ASP.NET Core MVC"* by Adam Freeman
- *"Microsoft SQL Server 2022: A Beginner's Guide"* by Dusan Petkovic
- *"Database System Concepts"* by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan

AI - Powered Mental Health Risk Assessment For Schizophrenia

Authors: ARUN GLADSON S (1), Co-Authors: ANGELIN JESIBA M (2), Dr M VIJAYA MAHESWARI (3), ARAVINDH K (4), MANICKARAJ T(5)

(1) (2) (4) (5): Student, Department: MCA, Name of the Institution: Francis Xavier Engineering College, University: Anna University, State: Tamil Nadu, Country: India

(3): Assistant Professor, Department: MCA, Name of the Institution: Francis Xavier Engineering College, University: Anna University, State: Tamil Nadu, Country: India

Email:aruns.pg.24.mca@francisxavier.ac.in,

angelinm.pg.24.mca@francisxavier.ac.in,vijaya@francisxavier.ac.in,

aravinthp.pg.25.mca@francisxavier.ac.in,

manickarajt.pg.25.mca@francisxavier.ac.in

ABSTRACT

Schizophrenia is a severe and chronic mental disorder that affects perception, thinking, behavior, and emotional regulation. Early identification of individuals at risk is crucial for timely intervention and improved treatment outcomes. Traditional diagnostic approaches primarily rely on clinical observation and subjective assessments, which may delay early detection. This study proposes an AI-powered mental health risk assessment framework for the early prediction of schizophrenia using machine learning techniques. The system integrates clinical, behavioral, demographic, and psychological assessment data to build predictive models capable of identifying high-risk individuals. Various supervised learning algorithms such as Logistic Regression, and Neural Networks are evaluated to determine optimal performance.The proposed model aims to enhance prediction accuracy, reduce misclassifications rates, and support clinicians in decision-making processes. Feature selection and data preprocessing techniques are applied to improve model reliability and generalization.

2nd International Conference on Recent Innovation in Life Sciences, Healthcare for Sustainable Development ICRILSH 2026

26th & 27th February 2026

Kuala Lumpur, Malaysia

Organized by

IFERP Life Sciences-Formerly BioLeagues

Academic Partner

Department of MCA, Francis Xavier Engineering College, India

ISBN : 978-93-92106-61-3

AI - Powered Mental Health Risk Assessment for Schizophrenia

Arun Gladson S
Student, Master of Computer Application, Francis Xavier Engineering College, Anna University, Tamil Nadu, India
Angelin Jesibam M
Student, Master of Computer Application, Francis Xavier Engineering College, Anna University, Tamil Nadu, India
Dr. M Vijaya Maheswari
Assistant Professor, Student, Master of Computer Application, Francis Xavier Engineering College, Anna University, Tamil Nadu, India
Aravindh K
Student, Master of Computer Application, Francis Xavier Engineering College, Anna University, Tamil Nadu, India
Manickaraj T
Student, Master of Computer Application, Francis Xavier Engineering College, Anna University, Tamil Nadu, India

Abstract:

Schizophrenia is a severe and chronic mental disorder that affects perception, thinking, behavior, and emotional regulation. Early identification of individuals at risk is crucial for timely intervention and improved treatment outcomes. Traditional diagnostic approaches primarily rely on clinical observation and subjective assessments, which may delay early detection.

This paper proposes an AI-powered mental health risk assessment for the early prediction of schizophrenia using machine learning techniques. The integrates clinical, behavioral, demographic, and psychological assessment data to build predictive models capable of identifying high-risk individuals. Various supervised learning algorithms such as Logistic Regression, and Neural Networks are evaluated to determine optimal performance.

The proposed model aims to enhance prediction accuracy, reduce misclassifications rates, and support clinicians in decision-making processes. Feature selection and data preprocessing techniques are applied to improve model reliability and generalization.

Keywords:

Schizophrenia, Artificial Intelligence (AI), Machine Learning, Mental Health Risk Assessment Early Detection, Predictive Modeling.